

CURSO AI PARA UXERS · WAVE CHARLIE · S6-S7

Git y GitHub para diseñadores.

Tu red de seguridad para construir con un agente. Hecho para diseñadores, no para desarrolladores: que nunca pierdas tu trabajo, que puedas volver atrás cuando un agente rompa algo, y que mañana publiques tu proyecto en una URL en vivo.

PARA

Diseñadores · sin saber programar

REGLA DE ORO

Tener siempre un punto al que volver

CAMINOS

Agente · GitHub Desktop · terminal

CONECTA CON

S6 respaldo · S7 despliegue en vivo

🔖 01 · QUÉ ES Y CÓMO USARLO

Git vs. GitHub (no son lo mismo)

	QUÉ ES	DÓNDE VIVE	ANALOGÍA
Git	El programa que guarda versiones de tu proyecto en tu computador	Tu máquina (local)	El "deshacer" infinito de tu proyecto
GitHub	La nube donde subes esas versiones para respaldarlas y compartirlas	Internet (remoto)	Google Drive, pero para código

💡 Tienes **dos fuentes de la verdad** — tu computador (local) y GitHub (nube). Git es el puente que las mantiene sincronizadas.

🎯 ¿POR QUÉ TE IMPORTA A TI (DISEÑADOR CON UN AGENTE)?

- **No pierdes trabajo.** El agente edita varios archivos a la vez. Si rompe algo, vuelves a la última versión buena en segundos.
- **Experimentas sin miedo.** Pruebas una feature; si no funciona, la descartas y quedas como estabas.
- **El agente trabaja mejor.** Un repo ordenado + tu carpeta `/contexto` = el agente entiende tu proyecto.
- **Es el requisito para desplegar.** En la S7, tu URL en vivo sale directo desde GitHub. Sin GitHub, no hay deploy.

🔗 LOS 6 CONCEPTOS QUE NECESITAS (Y NADA MÁS)

PALABRA	QUÉ SIGNIFICA EN CRISTIANO
Repositorio (repo)	La carpeta de tu proyecto que Git está vigilando
Commit	Una "foto" guardada de tu proyecto en un momento dado (un punto de retorno)
Push	Subir tus commits a GitHub (a la nube)
Pull	Bajar de GitHub los cambios que no tienes en tu máquina
Branch (rama)	Una línea paralela para experimentar sin tocar lo que funciona
Clone	Descargar un repo de GitHub a tu computador por primera vez

💡 El 90% del tiempo solo usarás **commit** (guardar foto) y **push** (subir a la nube).

🔄 EL CICLO QUE REPETIRÁS SIEMPRE

EL BUCLE DE TRABAJO

1. Trabajas (tú + el agente cambian archivos)
↓
2. COMMIT → "guardo una foto de cómo quedó" (con un mensaje claro)
↓
3. PUSH → "subo esa foto a GitHub"
↓
(repites desde 1 cada vez que algo funciona)

Eso es todo. **Commit** cuando algo funcione, **push** para respaldarlo.

 02 · INSTALACIÓN Y CONFIGURACIÓN (UNA SOLA VEZ)**A Crea tu cuenta de GitHub**

Entra a github.com → **Sign up**. Usa un correo que revises y un usuario sencillo y profesional (te servirá de portafolio). Verifica tu correo. El plan gratuito es más que suficiente para el curso.

B Instala Git

En Mac: abre la app **Terminal** y escribe `git --version`. Si no lo tienes, te ofrecerá instalar las "herramientas de línea de comandos" → acepta. (O desde `git-scm.com`.)

En Windows: descarga desde git-scm.com/download/win y ejecuta el instalador. Acepta las opciones por defecto (incluye **Git Bash**, la terminal donde correrás los comandos).

C Preséntate ante Git (nombre y correo)

Abre tu terminal (Terminal en Mac / Git Bash en Windows) y pega, con TUS datos:

TERMINAL · CONFIGURACIÓN GLOBAL

```
git config --global user.name "Tu Nombre"
git config --global user.email "tu-correo@ejemplo.com"
```

 Usa el **mismo correo** con el que te registraste en GitHub.

D Recomendado · Instala GitHub Desktop

Si la terminal te intimida, este es tu mejor amigo: una app **visual** para hacer commits y push con clics. Descárgala en desktop.github.com (Mac y Windows) e inicia sesión con tu cuenta de GitHub. Listo: la autenticación queda resuelta.

 Con esto ya estás listo. Tienes **3 caminos** para usar Git; elige el que te sea cómodo (página siguiente).

🔗 CÓMO USARLO EN TU PROYECTO — ELIGE TU CAMINO

1 El agente lo hace por ti (el más fácil)

Tu agente (Cursor, Claude, etc.) puede manejar Git solo. Pídeselo en lenguaje natural:

PROMPT · PRIMERA VEZ

Inicializa git en este proyecto, haz un primer commit con todo, crea un repositorio en mi GitHub y súbelo.

PROMPT · CADA VEZ QUE ALGO FUNCIONE

Haz commit de estos cambios con un mensaje claro y súbelos a GitHub.

2 GitHub Desktop (visual, sin comandos)

- 1 **File → Add Local Repository** → elige la carpeta de tu proyecto (o **Create a New Repository**).
- 2 Verás la lista de cambios. Escribe un **mensaje** abajo a la izquierda y clic en **Commit to main**.
- 3 Clic en **Publish repository** (primera vez) o **Push origin** (siguientes) para subirlo a GitHub.

3 La terminal (los comandos básicos)

DENTRO DE LA CARPETA DE TU PROYECTO

```
git init                # vigila este proyecto (solo la 1ª vez)
git add .               # selecciona todos los cambios
git commit -m "Primer commit" # toma la foto con un mensaje
```

CONECTAR CON GITHUB (CREA ANTES EL REPO VACÍO → BOTÓN NEW)

```
git remote add origin https://github.com/TU-USUARIO/TU-REPO.git
git branch -M main
git push -u origin main      # sube todo por primera vez
```

DE AHÍ EN ADELANTE, EL CICLO DE SIEMPRE

```
git add .
git commit -m "Agrego filtro de rutas al dashboard"
git push
```

 EL .GITIGNORE — ESTO TE AHORRA PROBLEMAS

No todo debe subir a la nube

Hay archivos que **NO** debes subir: pesados, privados o que se regeneran solos. Crea un archivo llamado `.gitignore` en la raíz del proyecto con:

```
.GITIGNORE
node_modules/
.env
.DS_Store
dist/
build/
*.log
```

QUÉ EVITAS SUBIR	POR QUÉ
node_modules/	Pesa cientos de MB y se reinstala solo con <code>npm install</code>
.env	Contiene claves secretas / API keys — ¡nunca a la nube!
.DS_Store	Basura del Finder de Mac
dist/ · build/	Se regeneran al compilar

 ¿Usas un agente? Pídele: "crea un .gitignore apropiado para este proyecto". Lo hace bien.

 03 · HAZLO ASÍ

- **Commits pequeños y frecuentes.** Cada vez que algo funcione, haz commit. Uno por feature o arreglo; no esperes al final del día.
- **Mensajes claros y en presente.** "Agrego estado vacío al dashboard" > "cambios", "asdf", "final final v2".
- **Haz push al terminar de trabajar.** Lo que no subes a GitHub solo vive en tu máquina (y se puede perder).
- **Commit ANTES de pedirle algo grande al agente.** Si lo rompe, vuelves al punto bueno al instante.
- **Una rama para experimentar.** Algo arriesgado: `git checkout -b prueba-x` (o "New Branch"). Si sale mal, descartas la rama y tu `main` queda intacto.
- **Tu repo es tu portafolio.** Buen nombre + un `README.md` que explique qué hace el proyecto.

 EVITA ESTO

- **NO subas secretos** (`.env` , API keys, contraseñas). Si lo haces por error, esa clave se considera comprometida: **rótala / cámbiala**.
- **NO subas node_modules** ni carpetas pesadas (usa `.gitignore`).
- **NO subas datos reales de usuarios** (recuerda el **D4** de tu Ficha 4D: datos mock, nunca reales).
- **No le tengas miedo a Git.** Casi todo es reversible. El peor error es no hacer commits y perder horas.
- **No trabajes horas sin guardar.** "Hago commit cuando termine todo" = receta para perderlo todo.

 **La mentalidad correcta:** Git es tu checkpoint de videojuego. Guarda seguido y juega tranquilo.

 PROBLEMAS COMUNES (Y CÓMO SALIR)

TE PASA ESTO	QUÉ HACER
"Rompí todo y quiero volver a mi último commit"	<code>git restore .</code> (descarta cambios sin commit). O en GitHub Desktop: clic derecho en los cambios → Discard changes
"Quiero volver a un commit anterior"	GitHub Desktop: pestaña History → clic derecho en el commit → Revert . O pídeselo al agente
"Me pide usuario y contraseña al hacer push y no funciona"	Usa GitHub Desktop (resuelve la autenticación solo) o configura un token. Pídele ayuda al agente
"Subí un archivo que no debía (ej: .env)"	Agrégallo al <code>.gitignore</code> , bórralo del repo y cambia esa clave secreta. Pide al agente el procedimiento exacto
"El push dice que el remoto tiene cambios"	Haz <code>git pull</code> primero, resuelve, y vuelve a <code>git push</code>

 Para casi cualquier enredo: **pega el mensaje de error a tu agente** y pídele que te explique en lenguaje sencillo qué hacer. Es la forma más rápida de aprender Git.

 CHECKLIST · ¿ESTÁS LISTO PARA LA CLASE?

- ☐ **Cuenta de GitHub** creada y verificada.
- ☐ **Git instalado** (`git --version` responde).
- ☐ Configuré mi **nombre y correo** en Git.
- ☐ (Recomendado) Instalé **GitHub Desktop** e inicié sesión.
- ☐ Sé hacer el ciclo mínimo: **commit** → **push**.
- ☐ Tengo un `.gitignore` (o sé pedirselo al agente).

→ CÓMO SE CONECTA CON EL CURSO

Hoy respaldas; la próxima, despliegas

Este instructivo es el puente entre dos sesiones. Lo que ordenas hoy es exactamente lo que la S7 necesita para sacar tu proyecto al mundo.

- **S6 (hoy):** ordenas tu repo y subes tu proyecto con su carpeta `/contexto` a GitHub. Tu trabajo queda respaldado y versionado.
- **S7 (próxima):** desde ese repo de GitHub **desplegamos tu proyecto a una URL en vivo**. Por eso hoy es clave dejar todo subido.

GLOSARIO EXPRESS

PALABRA	EN CRISTIANO
repo	Carpeta vigilada por Git
commit	Foto guardada (punto de retorno)
push	Subir a la nube (GitHub)
pull	Bajar de la nube a tu máquina
branch	Línea paralela para experimentar

Git = tu checkpoint de videojuego

Guarda seguido y juega tranquilo. Casi todo es reversible; el peor error es no hacer commits.

Interface School · AI para UXers · Wave Charlie · Instructivo Git y GitHub para diseñadores · S6-S7